PARAMETER ESTIMATION OF A DC MOTOR USING LINEAR LEAST SQUARES

1.0 Learning Outcomes 1
2.0 Hardware Set-Up 1
3.0 Practical Investigation via Simulink 2
4.0 Concluding Remarks 8
Recommended Citation 9

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Configure Simulink to log the applied voltage, the measured motor speed and time to the MATLAB workspace, and capture the open-loop step response of a DC motor.	☐
2	Use linear least squares to estimate a first-order discrete-time model of the motor, convert it into a continuous-time transfer function, and interpret its gain and time constant.	☐
3	Investigate how a discrete-time low-pass filter changes the measured data and the identified model, and validate each model by comparing the measured and simulated responses.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercise, DC Motor Speed Measurement using an Encoder, which details the operation of a DC motor with an encoder. The set-up measures the speed of the motor in revolutions per minute (RPM) and is not repeated here.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. A known voltage is applied to the motor in open loop, the input and output data are logged to the MATLAB workspace, and linear least squares (LLS) is used to estimate a mathematical model of the motor. The identified model is used in later practical exercises to build a simulation of the motor and to tune a PI controller before it is deployed to the hardware.

Log the Input, Output and Time Data

In the Simulink model, a Constant of 255 drives the PWM output so that 5V is applied to the motor, and a Gain of 5/255 converts this value into volts for logging, see Figure 1a. A Clock records the simulation time (Figure 1b), and the encoder measures the motor speed in RPM, see Figure 1c. The input, output and time data must be logged to the MATLAB workspace for the LLS script.

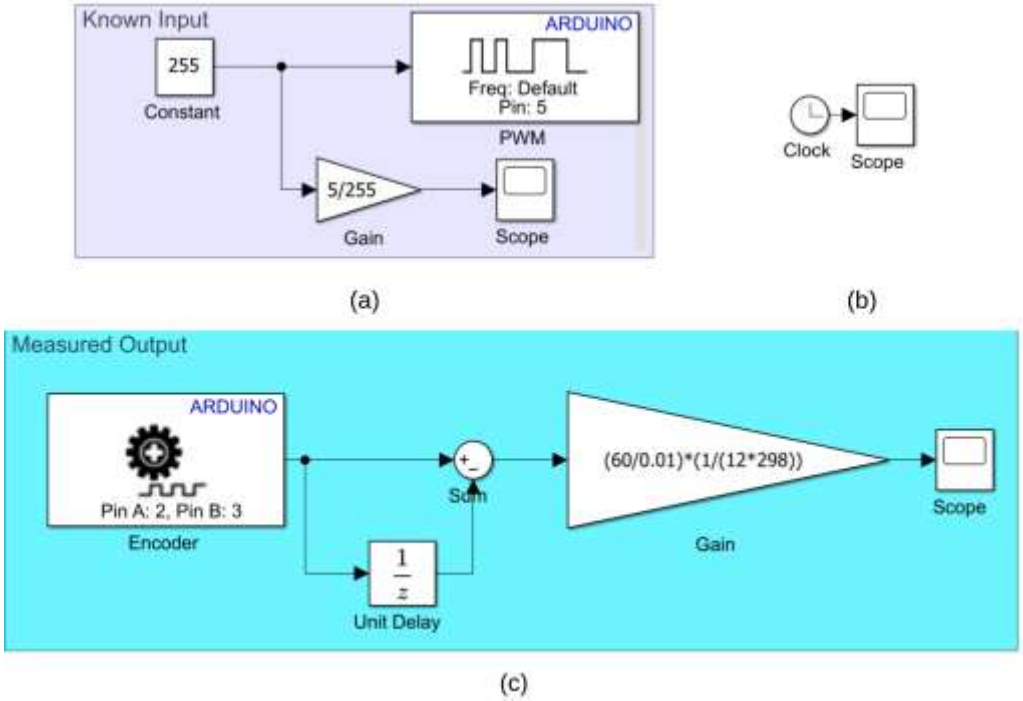


Figure 1: *Known Input Applied to the Motor (a), Time Recording (b), and the Measured Output Speed (c)*

1. Double-click the Scope connected to the applied voltage (after the Gain of 5/255) and click Settings. Expand the Logging section, tick Log data to workspace, set the Variable name to u and the Save format to Structure With Time, see Figure 2.
2. Repeat for the Scope connected to the measured speed, using the Variable name y.
3. Add a Clock connected to a new Scope (Figure 1b), and log it in the same way with the Variable name t.
4. Set the Stop Time to 5 s and run the model on the hardware. Confirm that u, y and t appear in the workspace.

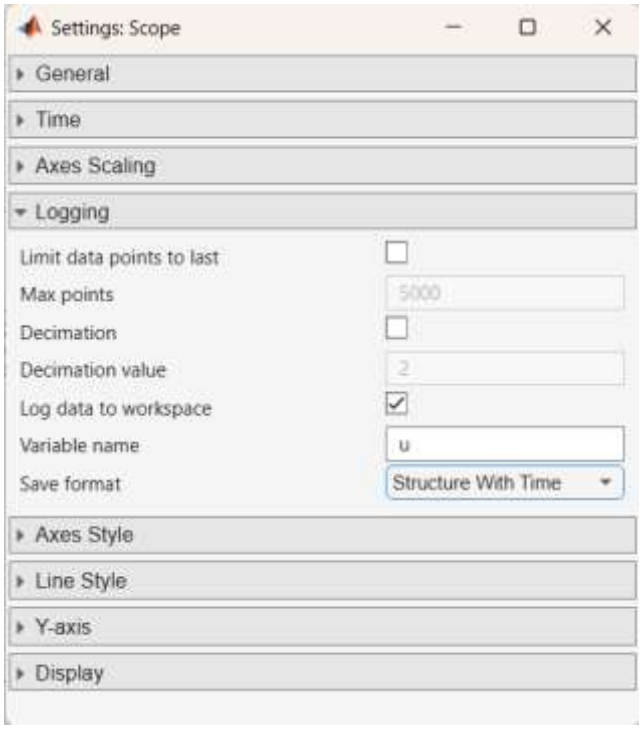


Figure 2: *Scope Logging Settings for the Input Voltage, u*

Select a Model Structure

The measured response to a 5V step is shown in Figure 3. The speed rises smoothly to approximately 125RPM without overshoot. Before continuing, decide whether a first- or second-order model is appropriate for your own response.

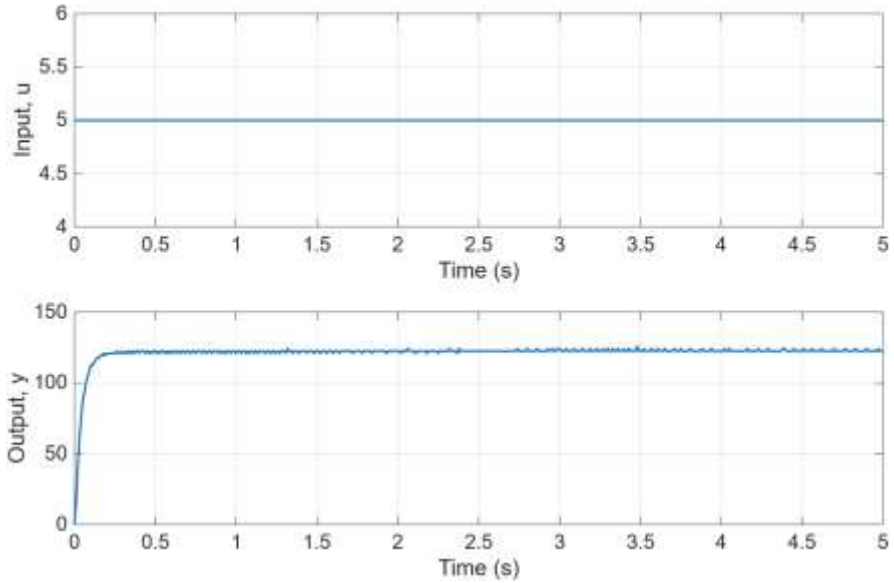


Figure 3: *Measured Input Voltage (Top) and Output Speed (Bottom) for a 5V Step*

A response without overshoot can be represented by a first-order model, written as the difference equation:

$$y_k = -a_1\, y_{k-1} + b_0\, u_{k-1}$$

where u_k is the applied voltage (V) and y_k the measured speed (RPM), both sampled every T_s seconds. Stacking this equation for every sample gives $y = \Phi\theta$, and the LLS estimate minimises the sum of the squared errors between the measured and predicted outputs:

$$\hat{\theta} = [\Phi^T\Phi]^{-1}\Phi^T y, \qquad \hat{\theta} = \begin{bmatrix} a_1 \\ b_0 \end{bmatrix}, \qquad \Phi = \begin{bmatrix} -y_{k-1} & u_{k-1} \\ -y_{k-2} & u_{k-2} \\ \vdots & \vdots \\ -y_{k-N} & u_{k-N} \end{bmatrix}, \qquad y = \begin{bmatrix} y_k \\ y_{k-1} \\ \vdots \\ y_{k-N+1} \end{bmatrix}$$

The estimated parameters define the discrete-time transfer function, which is converted into a continuous-time transfer function using `d2c` in MATLAB. The continuous-time form gives the steady-state gain K (RPM/V) and the time constant τ (s):

$$G(z) = \frac{b_0}{z + a_1} \qquad \rightarrow \qquad G(s) = \frac{K}{\tau s + 1}$$

Estimate a First-Order Model of the Motor

The MATLAB script below estimates the parameters of the first-order difference equation using LLS. It then simulates the estimated model from the measured voltage alone, compares the simulated and measured speeds, and calculates the mean error and the mean absolute error (MAE). Finally, it converts the model into discrete- and continuous-time transfer functions and calculates K and τ.

MATLAB Script: estimating a first-order model of the DC motor using LLS

```
clear; clc; close all;

%% Load the logged data (Scope save format: Structure With Time)
u_k = u.signals.values(:);          % input voltage [V]
y_k = y.signals.values(:);          % measured motor speed [RPM]
t_k = t.signals.values(:);          % time [s]
Ts  = 0.01;                         % sample time [s]

%% Linear least squares estimate of a first-order model
Phi   = [-y_k(1:end-1), u_k(1:end-1)];   % observation matrix
Y     = y_k(2:end);                      % output vector
theta = (Phi'*Phi) \ (Phi'*Y);         % theta = [a1; b0]
a1 = theta(1);  b0 = theta(2);

%% Simulate the estimated model from the measured input only
N = length(y_k);  y_sim = zeros(N,1);
for k = 2:N
    y_sim(k) = -a1*y_sim(k-1) + b0*u_k(k-1);
end
e = y_k - y_sim;                          % modelling error [RPM]
MeanError = mean(e);  MAE = mean(abs(e));

%% Measured and simulated outputs, and the modelling error
subplot(211)
plot(t_k,u_k,'r', t_k,y_sim,'b', t_k,y_k,'k.'); grid on;
legend('Input voltage, u [V]','Simulated motor speed, y_{sim} [RPM]',
...
       'Measured motor speed, y [RPM]'); xlabel('Time [s]');
ylabel('Signal');
subplot(212)
plot(t_k,e); grid on; xlabel('Time [s]'); ylabel('Error [RPM]');
title(sprintf('Mean Error = %.3g RPM  |  MAE = %.3g
RPM',MeanError,MAE));

%% Discrete- and continuous-time transfer functions
Gd  = tf(b0,[1 a1],Ts)                    % discrete-time model
G   = d2c(Gd)                             % continuous-time model
den = G.Denominator{1};  num = G.Numerator{1};
tau = 1/den(2)                            % time constant [s]
K   = num(2)/den(2)                       % system gain [RPM/V]
```

Example results from an ACE-Lab motor are shown in Figure 4. The simulated speed follows the measured speed closely, with a mean error close to zero and an MAE of 0.375 RPM. The identified discrete- and continuous-time models are:

$$G(z^{-1}) = \frac{1.81z^{-1}}{1 - 0.936z^{-1}}, \qquad G(s) = \frac{187}{s + 6.67} = \frac{28}{0.15s + 1}\left[\frac{\text{RPM}}{\text{V}}\right]$$

giving $K \approx 28$ RPM/V and $\tau \approx 0.15$ s. Recall that the response reaches 63.2% of its final value after 1τ, 95.0% after 3τ and 99.3% after 5τ. Check that the identified K and τ agree with your measured step response.

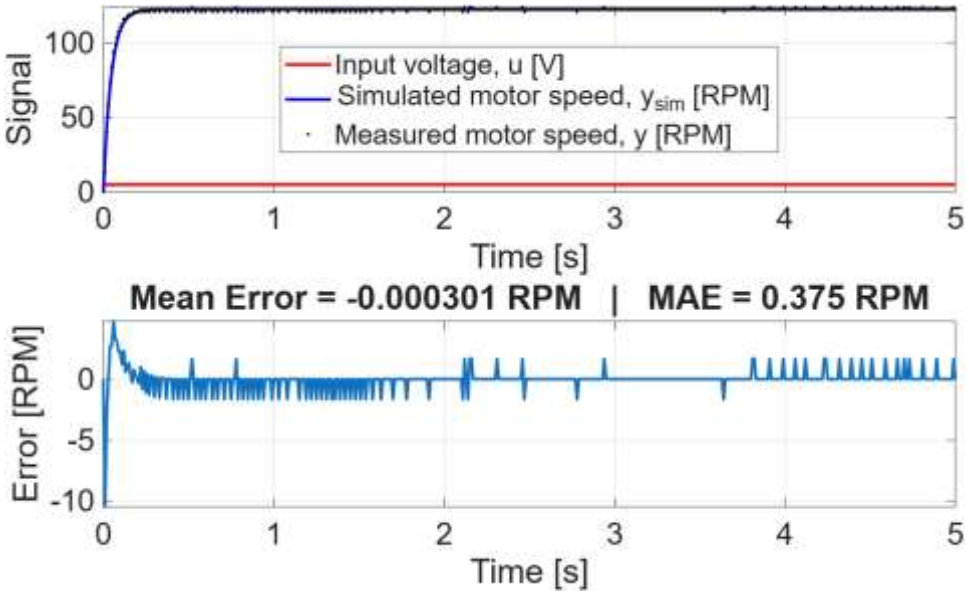


Figure 4: *Measured and Simulated Motor Speed (Top), and the Modelling Error (Bottom), with No Filtering*

Investigate the Effect of a Low-Pass Filter

Encoder speed measurements contain quantisation noise and occasional spikes, which are usually reduced with a discrete-time low-pass filter, as covered in the practical exercise, Discrete-Time Low-Pass Filter Design for DC Motor Speed Measurement. A first-order filter with time constant τ_f is converted into discrete time using the script below:

MATLAB Script: designing a first-order discrete-time low-pass filter

```
clear; clc; close all;
tau = 0.1;                      % filter time constant [s]
H   = tf(1,[tau 1]);            % continuous-time transfer function
T_s = 0.01;                     % sample time [s]
Hd  = c2d(H,T_s,'zoh');         % discrete-time transfer function
disp('Continuous-time model:'); H
disp('Discrete-time model:');   Hd
```

1. Add the discrete-time filter to the Simulink model after the measured speed, and log the unfiltered speed (y_u) and the filtered speed (y) to the workspace.
2. Repeat the test and identification with $\tau_f = 0.1$ s, for which the filter is $0.09516/(1 - 0.9048z^{-1})$, see Figure 5a.
3. Repeat with $\tau_f = 0.5$ s, for which the filter is $0.0198/(1 - 0.9802z^{-1})$, see Figure 5b.
4. Record the identified K, τ, mean error and MAE for each case in Table 3, and compare them with the example results in Table 2.

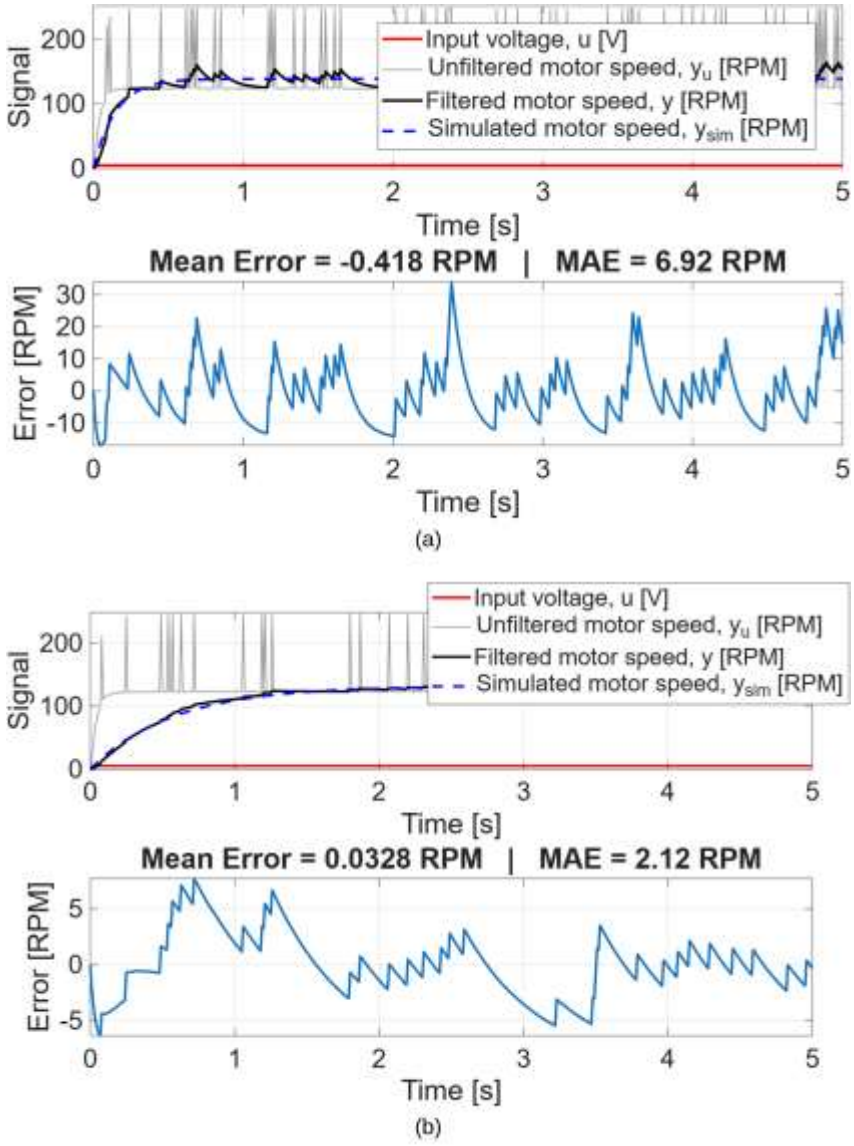


Figure 5: *Measured and Simulated Responses with a Low-Pass Filter Time Constant of 0.1 s (a) and 0.5 s (b)*

Table 2: *Example Results from an ACE-Lab Motor*

Filter	**Identified** $G(s)$	K **(RPM/V)**	τ **(s)**	**MAE (RPM)**
None	$187/(s+6.67)$	28	0.15	0.375
$\tau_f = 0.1$s	$188.5/(s+6.8)$	27.7	0.15	6.92
$\tau_f = 0.5$s	$47.3/(s+1.82)$	26	0.55	2.12

Table 3: *Identified Models for Your Motor*

Filter	Identified $G(s)$	K (RPM/V)	τ (s)	MAE (RPM)
None				
$\tau_f = 0.1$ s				
$\tau_f = 0.5$ s				

With $\tau_f = 0.1$ s the identified gain and time constant are almost unchanged, because the filter is fast compared with the motor. With $\tau_f = 0.5$s the identified time constant increases to about 0.55s: the model now describes the motor and the filter together, and the prime in $G'(s)$ is often used to indicate this. Explain the differences in your own results, and decide which model best represents the system that the controller will see.

Use the Identified Model in Simulation

Once you are satisfied with the low-pass filter design, identify the final model of the DC motor. Because the measured speed used by the controller passes through the same filter, the identified transfer function contains the dynamics of both the motor and the low-pass filter. The model can then be transferred into a Simulink simulation, incorporating as much of the practical control system behaviour as possible, such as actuator saturation, before tuning the PI controller, see Figure 6.

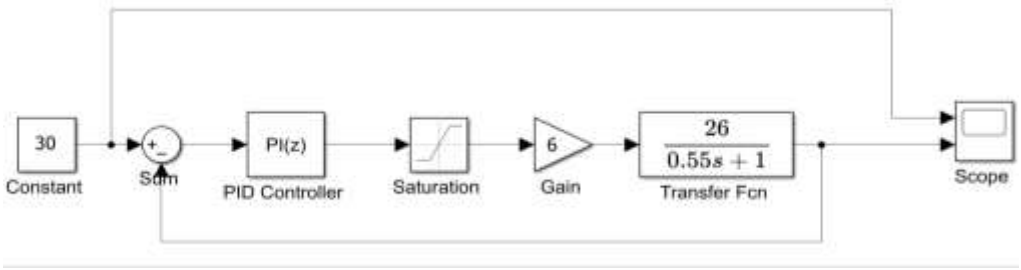


Figure 6: *Simulink Simulation of the Closed-Loop Speed Control System using the Identified Model*

4.0 Concluding Remarks

This practical exercise identified a mathematical model of a DC motor from measured input and output data. Using MATLAB, Simulink and ACE-Lab hardware, the applied voltage, measured speed and time were logged to the workspace, and linear least squares was used to estimate a first-order discrete-time model, which was converted into a continuous-time transfer function with a physically meaningful gain and time constant.

This practical exercise also showed that a low-pass filter changes the identified model: a fast filter has little effect, whereas a slower filter adds lag that appears

in the identified time constant. Validating each model against the measured response, and identifying the model with the same filter that the controller will use, gives a model that can be used with confidence to design and tune controllers in simulation.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Parameter Estimation of a DC Motor using Linear Least Squares. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/XXXXXXXX